Week 7

Classification

Guan-Yun Wang
Department of Information Management
Fu Jen Catholic University
2026.04.10

Introduction

- Classifying texts is a classic NLP problem.
- This NLP task involves assigning a value to a text
  - For example, a topic (such as sport or business) or a sentiment, such as negative or positive, and any such task needs evaluation.
- The supervised algorithms in spaCy framework:
  - support vector machines (SVMs)
  - convolutional neural network (CNN)

Today's content:

- Getting the dataset and evaluation ready
- Performing rule-based text classification using keywords
- Clustering sentences using K-Means - unsupervised text classification
- Using SVMs for supervised text classification
- Training a spaCy model for supervised text classification
- Classifying texts using LLM models

Preparation

1. Import necessary classes. Here we import the `detecct` function from `langdetect` , which will help us determine the language of the review.
   (`langdetect` supports 55 languages (ISO 639-1 codes)(including `zh-tw`))
2. We also import `word_tokenize` function, which will be used to split the reviews into words.
3. `FreqDist` is a `class` from NLTK, which will help us see the most frequent positive and negative words in the reviews.
4. The stop word list is from NLTK's `stopwords`
5. The `punctuation` string from the `string` package will help us to filter punctuation. (標點符號)

```
# !pip install langdetect # If it's your first time to use this
from langdetect import detect
from nltk import word_tokenize
from nltk.probability import FreqDist
from nltk.corpus import stopwords
from string import punctuation
```

Data

Loading the test datasets and print the two dataframes.

- We can see the data contains a `text` column and a `label` column

```
(train_df, test_df) = load_train_test_dataset_pd("train", "test")
print(train_df)
print(test_df)
```

The output of `train_df` should be like this:

```
text label
0 the rock is destined to be the 21st century's ... 1
1 the gorgeously elaborate continuation of " the... 1
2 effective but too-tepid biopic 1
3 if you sometimes like to go to the movies to h... 1
4 emerges as something rare , an issue movie tha... 1
... ... ...
8525 any enjoyment will be hinge from a personal th... 0
8526 if legendary shlockmeister ed wood had ever ma... 0
8527 hardly a nuanced portrait of a young woman's b... 0
8528 interminably bleak , to say nothing of boring . 0
8529 things really get weird , though not particula... 0
[8530 rows x 2 columns]
```

Data preprocessing

- Here we create a new column called `lang` in the dataframes that will contain the language of the review.
- Then, use the `detect` function to populate this column via the `apply` method.
- Here we also filter the dataframe to only contain English-language review.

```
# Filter out non-English text (training dataset)
train_df["lang"] = train_df["text"].apply(detect)
train_df = train_df[train_df['lang'] == 'en']
print(train_df)
# Filter out non-English text (testing dataset)
test_df["lang"] = test_df["text"].apply(detect)
test_df = test_df[test_df['lang'] == 'en']
print(test_df)
```

How many rows here?

Tokenization

- Here we tokenize the text into words.
  (We will use `english.pickle` tokenizer.)

```
# Split into words
train_df["tokenized_text"] = train_df["text"].apply(word_tokenize)
print(train_df)
test_df["tokenized_text"] = test_df["text"].apply(nltk.tokenize.word_tokenize)
print(test_df)
```

Stop words processing

- In this step, we remove stopwords and punctuation.
- First loading the `stopwords` datasets using NLTK package. Furthermore, add ``` and `'s` to the list of stopwords. (You can also add other stopwords that you need.)
- We then define a function that will take a list of words as input and filter it. Then return the new list that doesn't contain stopwords or punctuation.
- Apply this function to the training and test data.

```
nltk.download('stopwords')
# Remove stopwords and punctuation
stop_words = list(stopwords.words('english'))
stop_words.append("``")
stop_words.append("'s")
def remove_stopwords_and_punct(x):
    new_list = [w for w in x if w not in stop_words and w not in punctuation]
    return new_list
train_df["tokenized_text"] = train_df["tokenized_text"].apply(remove_stopwords_and_punct)
print(train_df)
test_df["tokenized_text"] = test_df["tokenized_text"].apply(remove_stopwords_and_punct)
print(test_df)
```

Class balance and save the datasets

Here we check the class balance over both datasets (training and testing dataset)

```
# Count number of items per class
print(train_df.groupby('label').count())
print(test_df.groupby('label').count())
```

If you don't want to lose your dataset that you've already finished the preprocessing.

Remember to save them!

```
# Save to disk
train_df.to_json("../data/rotten_tomatoes_train.json")
test_df.to_json("../data/rotten_tomatoes_test.json")
```

Examine the dataset

- We define a function here that will take a list of words and the number of words as input and return a `FreqDist` object.
- It will print out the top *n* most frequent words, where *n* is passed in to the function and `num_words=200` is default.

```
def get_stats(word_list, num_words=200):
    freq_dist = FreqDist(word_list)
    print(freq_dist.most_common(num_words))
    return freq_dist
```

- Now let's use the preceding function and show the most common words in positive and negative reviews to see whether there are significant vocabulary differences between the two classes.

```
# Show most common words
positive_train_words = train_df[train_df["label"] == 1].tokenized_text.sum()
negative_train_words = train_df[train_df["label"] == 0].tokenized_text.sum()
positive_fd = get_stats(positive_train_words)
negative_fd = get_stats(negative_train_words)
```

Q: What is the most frequent word in the training dataset?

Performing rule-based text classification using keywords

- This section we're using the vocabulary of the text to classify the Rotten Tomatoes reviews.
- We will create a simple classifier that will have a vectorizer for each class.
- The vectorizer will include the words characteristic to that class.
- The classification will simply be vectorizing the text using each of the vectorizers and then using the class that has more words.

Implementation

The necessary imports

```
from nltk import word_tokenize
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import classification_report
```

Load the cleaned data from disk. (Or maybe you don't restart your python file, you can skip this step)

```
# Load cleaned data from disk (If you start here after you open this file)
train_df = pd.read_json("../data/rotten_tomatoes_train.json")
test_df = pd.read_json("../data/rotten_tomatoes_test.json")
```

Preprocessing before classification

- Create a list of words unique to each class.
- Firstly, concatenate all the words from the `text` column, filtering on the relevant `label` value.
  - 0 for negative reviews and 1 for positive ones
- Get the words that appear in both of those lists in the `word_intersection` variable.
- Create filtered word lists, one for each class, that do not contain words that appear in both classess.

```
# Create the positive and negative word lists
# Only from training data
positive_train_words = train_df[train_df["label"] == 1].tokenized_text.sum()
negative_train_words = train_df[train_df["label"] == 0].tokenized_text.sum()
word_intersection = set(positive_train_words) & set(negative_train_words)
positive_filtered = list(set(positive_train_words) - word_intersection)
negative_filtered = list(set(negative_train_words) - word_intersection)
```

Create vectorizers

- Define a function to create vectorizers and the input of this function is a list of lists.
- For each of the word lists, use `CountVectorizer` object to take the word list as the `vocabulary` parameter.
- You can also find other vectorizer from `sklearn`
    - For example, _____, _____.

```
def create_vectorizers(word_lists):
    vectorizers = []
    for word_list in word_lists:
        vectorizer = CountVectorizer(vocabulary=word_list)
        vectorizers.append(vectorizer)
    return vectorizers
```

Therefore, we can create vectorizers by using the function we created.

```
vectorizers = create_vectorizers([negative_filtered, positive_filtered])
```

- Here we further create a `vectorize` function that takes in a list of words and a list of vectorizers.
- We first create a string from the word list, as the vectorizer expects a string.
- For each vectorizer in the list, we apply it to the text and then sum the total count of words in that vectorizer.
- We finally append that sum to a slit of scores.

```
def vectorize(text_list, vectorizers):
    text = " ".join(text_list)
    scores = []
    for vectorizer in vectorizers:
        output = vectorizer.transform([text])
        output_sum = sum(output.todense().tolist()[0])
        scores.append(output_sum)
    return scores
```

- Then, the `classify` function takes a list of socres returned by the `vectorize` function.
- The function simply selects the maximum score from the list and returns the index of that score corresponding to the class label.

```
def classify(score_list):
    return max(enumerate(score_list),key=lambda x: x[1])[0]
```

- After that, we can apply the functions to the training data.

```
train_df["prediction"] = train_df["tokenized_text"].apply(lambda x: classify(vectorize(x, vectorizers)))
print(train_df)
```

Metrics

The results report:

Training metrics

```
print(classification_report(train_df['label'], train_df['prediction']))
```

Test metrics

- Test the data on testing dataset.
- Measure the performance of the rule-based classifier by printing the classification report.

```
test_df["prediction"] = test_df["tokenized_text"].apply(lambda x: classify(vectorize(x, vectorizers)))
print(classification_report(test_df['label'], test_df['prediction']))
```

Clustering sentences using K-means - unsupervised text classification

- In this example, we will use the BBC news dataset.
  - The dataset contains news pieces sorted by five topics: politics, tech, business, sport, and entertainment.
- Unsupervised K-Means algorithm -> Sort the data into unlabeled classes

The necessary functions and packages

```
from nltk import word_tokenize
from sklearn.cluster import KMeans
from nltk.probability import FreqDist
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import StratifiedShuffleSplit
from joblib import dump, load
```

Load the dataset

- We're loading the BBC dataset. We use `load_dataset` function from Hugging Face's `datasets` package.
- In the Hugging Face repository, datasets are usually split into training and testing.
- Although we load both, in unsupervised learning, the test set is usually not used.

```
# Load dataset
train_dataset = load_dataset("SetFit/bbc-news", split="train")
test_dataset = load_dataset("SetFit/bbc-news", split="test")
train_df = train_dataset.to_pandas()
test_df = test_dataset.to_pandas()
print(train_df)
print(test_df)
```

Examine the data

Check the distribution of items per class for both training and test data.

- Class balance is important in classification
  - a disproportionally larger class will influence the final classifier (WHY?)

```
# See the distribution of classes
print(train_df.groupby('label_text').count())
print(test_df.groupby('label_text').count())
```

Prepare a new train/test split

Since there is almost as much data in the test set as in the training set, we will combine the data and create a better train/test split.

```
# Combine train and test dataframes and create a better train/test split
combined_df = pd.concat([train_df, test_df], ignore_index=True, sort=False)
sss = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=0)
train_index, test_index = next(sss.split(combined_df["text"], combined_df["label"]))
train_df = combined_df[combined_df.index.isin(train_index)].copy()
test_df = combined_df[combined_df.index.isin(test_index)].copy()
print(train_df.groupby('label_text').count())
print(test_df.groupby('label_text').count())
```

Word Preprocessing

- Tokenize the data and remove stopwords and punctuation.
- The function is defined in advanced (See your notebook, the "clean data" section)

```
# Preprocess the data
train_df = tokenize(train_df, "text")
train_df = remove_stopword_punct(train_df, "text_tokenized")
test_df = tokenize(test_df, "text")
test_df = remove_stopword_punct(test_df, "text_tokenized")
print(train_df)
print(test_df)
```

Create vectorizer and save the clean text

- Here we create the vectorizer. (TF-IDF vectorizer)
  - The TF-IDF vectorizer will count unigrams, bigrams, and trigrams (use the `ngram_range` parameter)
- You can notice that we only fit the vectorizer on the training data.
  - Because if we fit it on both training and test data, it would lead to data leakage and we would get better test scores than actual performance on unseen data.

```
# Get the training data and create the vectorizer
train_df["text_clean"] = train_df["text_tokenized"].apply(lambda x: " ".join(list(x)))
test_df["text_clean"] = test_df["text_tokenized"].apply(lambda x: " ".join(list(x)))
train_df.to_json("../data/bbc_train.json")
test_df.to_json("../data/bbc_test.json")
vec = TfidfVectorizer(ngram_range=(1,3))
matrix = vec.fit_transform(train_df["text_clean"])
```

`Kmeans` classifier

- Create a `Kemans` classifier for five clusters and then fit it on the matrix produced using the vectorizer from the preceding code.
  - `n_clusters` parameter is the number of clusters
  - `n_init` parameter is the number of times the algorithm should run
    - (For higher-dimensional problems, it is recommended to do several runs.)

```
# Cluster the data
km = KMeans(n_clusters=5, n_init=10)
km.fit(matrix)
```

This function will return a list of the most frequent words in a list. This will provide us with a clue as to which topic the text is about.

```
def get_most_frequent_words(text, num_words):
    word_list = word_tokenize(text)
    freq_dist = FreqDist(word_list)
    top_words = freq_dist.most_common(num_words)
    top_words = [word[0] for word in top_words]
    return top_words
```

This function use the `get_most_frequent_words` function.

It takes the dataframe, the `KMeans` model, and the number of clusters as input parameters.

This function will return the input dataframe with the added cluster number column

```
def print_most_common_words_by_cluster(input_df, km, num_clusters):
    clusters = km.labels_.tolist()
    input_df["cluster"] = clusters
    for cluster in range(0, num_clusters):
        this_cluster_text = input_df[input_df['cluster'] == cluster]
        all_text = " ".join(this_cluster_text['text_clean'].astype(str))
        top_200 = get_most_frequent_words(all_text, 200)
        print(cluster)
        print(top_200)
    return input_df
```

Check the results

Check out the most common words by different clusters:

```
print_most_common_words_by_cluster(train_df, km, 5)
```

Can you identify the most common words of each cluster?

Can you name the cluster?

Here, we test the KMean model on testing dataset

```
test_example = test_df.iloc[1, test_df.columns.get_loc('text')]
print(test_example)
vectorized = vec.transform([test_example])
prediction = km.predict(vectorized)
print(prediction)
```

Are you satisfied with the prediction?

Using SVMs for supervised text classification

Here we build a machine learning classifier that uses the SVM algorithm.

- In this example, we also use BBC news datasets

The necessary functions and packages:

```
from sklearn.svm import SVC
from sentence_transformers import SentenceTransformer
from sklearn.metrics import confusion_matrix
```

Load the training and test data

- You have already cleaned the BBC news dataset in the previous section.

```
train_df = pd.read_json("../data/bbc_train.json")
test_df = pd.read_json("../data/bbc_test.json")
train_df.sample(frac=1)
print(train_df.groupby('label_text').count())
print(test_df.groupby('label_text').count())
```

Here, we load the sentence transformer `all-MiniLM-L6-V2` model that will provide the vectors for us.
We then define the `get_sentence_vector` function, which returns the sentence embedding for the text input:

```
model = SentenceTransformer('all-MiniLM-L6-v2')
def get_sentence_vector(text, model):
    sentence_embeddings = model.encode([text])
    return sentence_embeddings[0]
```

And define the classifier:

```
def train_classifier(X_train, y_train):
    clf = SVC(C=0.1, kernel='rbf')
    clf = clf.fit(X_train, y_train)
    return clf
```

Test your model

We create training and test datsets and train the classifier using the `train_classifier` function.

After that, we print the training and test metrics.

```
target_names=["tech", "business", "sport", "entertainment", "politics"]
vectorize = lambda x: get_sentence_vector(x, model)
(X_train, X_test, y_train, y_test) = create_train_test_data(train_df, test_df, vectorize, column_name="text_clean")
clf = train_classifier(X_train, y_train)
print(classification_report(train_df["label"], y_train, target_names=target_names))
test_classifier(test_df, clf, target_names=target_names)
```

Confusion matrix

```
print(confusion_matrix(test_df["label"], test_df["prediction"]))
```

Results:

```
[[177 8 2 2 0]
 [ 7 210 4 0 3]
 [ 0 0 235 0 1]
 [ 2 1 0 172 1]
 [ 1 6 1 0 167]]

(tech, business, sport, entertainment, politics)
```

Test the classifier on a new example

```
new_example = """
iPhone 12: Apple makes jump to 5G
Apple has confirmed its iPhone 12 handsets
will be its first to work on faster 5G networks.
.......
This has been linked to reports that several Chinese internet platforms
opted not to carry the livestream,
although it was still widely viewed and commented
on via the social media network Sina Weibo.
"""

vector = vectorize(new_example)
prediction = clf.predict([vector])
print(prediction)
```

Classifying texts using LLM models

You can choose any LLM you prefer. In this example, we use Mistral AI's model.

The model we use here is "mistral-small-2506", but you can use the latest model. Or use another specific model for you own purpose.

Before you start, make sure you have an API key. (For any LLM)

```
MISTRAL_API_KEY="You need to use your own!!~~"
```

The necessary functions and packages:

```
import re
from sklearn.metrics import classification_report
from mistralai.client import Mistral
client = Mistral(api_key=MISTRAL_API_KEY)
model = "mistral-small-2506"
```

Load the training and test datasets using Hugging Face without preprocessing them for the number of classes.

```
train_dataset = load_dataset("SetFit/bbc-news", split="train")
test_dataset = load_dataset("SetFit/bbc-news", split="test")
```

Examine the first example

```
example = test_dataset[0]["text"]
category = test_dataset[0]["label_text"]
print(example)
print(category)
```

Run the LLM model on the first example.

```
# Run "mistral-small-2506" for one example
prompt="""You are classifying texts by topics. There are 5 topics: tech, entertainment, business, politics and sport.
Output the topic and nothing else. For example, if the topic is business, your output should be "business".
Give the following text, what is its topic from the above list without any additional explanations: """ + example
response = client.chat.complete(
    model=model,
    temperature=0,
    max_tokens=256,
    top_p=1.0,
    frequency_penalty=0,
    presence_penalty=0,
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": prompt}
    ],
)
print(response.choices[0].message.content)
```

Is this the same as the example of the previous page?

Create a LLM classification function

```
# Evaluate Mistral classification on several examples
def get_mistral_classification(input_text):
    prompt="""You are classifying texts by topics. There are 5 topics: tech, entertainment, business, politics and sport.
Output the topic and nothing else. For example, if the topic is business, your output should be "business".
Give the following text, what is its topic from the above list without any additional explanations: """ + input_text
    response = client.chat.complete(
        model=model,
        temperature=0,
        max_tokens=256,
        top_p=1.0,
        frequency_penalty=0,
        presence_penalty=0,
        messages=[
            {"role": "system", "content": "You are a helpful assistant."},
            {"role": "user", "content": prompt}
        ],
    )
    classification = response.choices[0].message.content
    classification = classification.lower().strip()
    return classification
```

Load the test data

- Shuffle the dataframe and select the first _____ examples.
  - Here we tested 50 examples. If you don't have limitation on tokens, you can test more example, such as 200 examples.
- Always think about the cost of using LLM's API, you can modify how much data you test this method on:

```
test_df = test_dataset.to_pandas()
test_df.sample(frac=1)
test_data = test_df[0:50].copy()
```

It will take a few minutes to run:

```
test_data["gpt_prediction"] = test_data["text"].apply(lambda x: get_mistral_classification(x))
```

Check the results:

```
def get_one_word_match(input_text):
    loc = re.search(r'tech|entertainment|business|sport|politics', input_text).span()
    return input_text[loc[0]:loc[1]]
test_data["gpt_prediction"] = test_data["gpt_prediction"].apply(lambda x: get_one_word_match(x))
```

Check the results:

```
label_list = ["tech", "business", "sport", "entertainment", "politics"]
test_data["mistral_label"] = test_data["mistral_prediction"].apply(lambda x: label_list.index(x))
print(test_data)
```

Check the metrics:

```
print(classification_report(test_data["label"], test_data["gpt_label"], target_names=label_list))
```